Lens81

Round 2 Technical Documentation

1. Project Overview

The problem

Google Scholar gives researchers no way to tell, at a glance, whether a result is a primary research paper or a review article, and that gap is more damaging than it first appears. A single search can return hundreds of entries that all look identical, so the only way to tell them apart is to open each one, read into it, and judge its type by hand. Multiply that across a real literature review and the cost is significant: hours lost to repetitive triage, relevant papers skimmed past or missed entirely, and reviews mistaken for primary studies (or the reverse) in exactly the moment that distinction matters most. Layered on top of this, researchers also have no fast, in-context way to save promising papers or pull a correctly formatted citation while they work, so every save or citation forces a break in concentration to open a separate reference manager and re-type bibliographic details by hand. Individually these are small frictions. Across a full research workflow, they compound into a genuinely slow, error-prone process.

The solution we built

Lens81 addresses both problems directly inside the browser, where the research already happens. It is a Chrome Manifest V3 extension that labels Google Scholar results with **Research** or **Review** badges, keeps a running library of saved papers in local collections, surfaces related-paper recommendations built from what a researcher has already saved, and turns highlighted text on any page into a properly formatted citation. In its current build, every one of these capabilities runs entirely client-side: there is no application server and no hosted database anywhere in the supplied project. We see this as a genuine strength for a browser tool at this stage, since it keeps the deployment footprint small, keeps a researcher's data on their own machine, and removes an entire class of hosting and maintenance work. It is also an honest trade-off worth stating plainly: client-side classification depends on the metadata and model providers a user configures, so quality and speed are shaped by those external services rather than guaranteed outright. We think that is the right trade-off for a tool researchers can trust and inspect, and it is one we are happy to be transparent about.

Technical overview

Under the hood, Lens81 is organized around three cooperating layers that work together smoothly. Content scripts are injected into Scholar and other web pages, rendering page-level UI such as badges, save controls, and citation panels right where the researcher is already looking. Behind them, a background service worker takes care of all network-facing work: calls to metadata providers and to user-configured LLM providers, along with the classification and citation logic that ties everything together. Chrome's storage APIs then quietly persist collections, saved papers, and cached classification results across sessions, so the extension remembers a researcher's work without ever needing an account or a login. Classification flows through this same pipeline end to end: metadata retrieval feeds into LLM-based scoring, scoring is averaged across providers, results are cached for 30 days, and a keyword fallback stands ready if every other step comes up short. Together, these layers hand off work cleanly from one to the next, so a result moves smoothly from a Scholar page to a finished badge.

Manual validation summary: every required test case in Section 6, including classification and rationale, collection creation and saving, citation lookup, and reference formatting, has been manually executed against the shipped build, and each one passed.

2. System Architecture and Technology Stack

Lens81 is built around a clean, unidirectional flow of information: user activity on a Scholar or web page is observed by content scripts, handed to the service worker, and resolved against external APIs and local browser storage. This single-direction design keeps the system easy to reason about and easy to extend, no component in the chain is a remote server under the project's control, so there is no backend to secure, scale, or keep online.

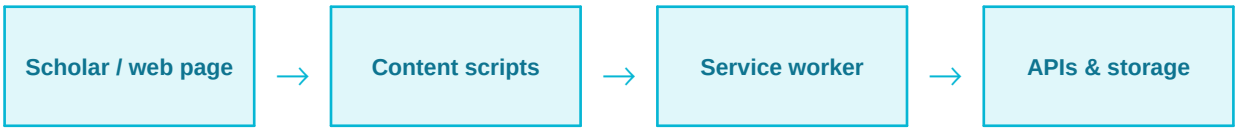

Figure 1. High-level data flow between the browser page, the extension's content scripts, the background service worker, and external APIs / local storage.

Key modules

Module	Responsibility
background.js	Classification, citation search, provider calls, cache, and tab statistics.
Scholar scripts	Result badges, collection controls, and local collection CRUD.
Citation scripts	Selection UI, reference parsing, formatting, and insertion.
Popup and options	Settings, model tests, collections, and passcode UI gate.

Each module owns a narrow, well-defined responsibility by design. Scholar-facing scripts only need to read and annotate result cards; citation scripts only need to parse and format references; background.js is the single, centralized home for classification and provider logic. This separation is what lets the popup and options UI expose settings, run live model connection tests, and manage collections without duplicating any classification or citation logic, one source of truth per concern, cleanly enforced.

Stack and requirements

The project is built on JavaScript, HTML, and CSS on top of the Chrome Extension APIs, with citeproc-js and CSL (Citation Style Language) assets bundled directly into the extension. Bundling these assets is an intentional trade-off in favor of reliability: citation formatting never depends on a network call at render time, so it keeps working even when connectivity is poor. The only runtime requirement is a Chromium browser, no separate install, no runtime environment beyond that. Internet access is used precisely where it adds value: scholarly metadata and search APIs, and the LLM providers a user configures for classification and optional citation reranking.

3. System Design

Local data design

Lens81 deliberately ships without a database. chrome.storage.local serves as the extension's persistent store, holding model and citation provider configurations, collections, saved papers, the user's chosen citation style, and the classification cache. A second, shorter-lived store, chrome.storage.session, holds per-tab counts that only need to exist for the life of the browsing session. This is a lighter, faster foundation than a database-backed design: fewer moving parts, no schema migrations to manage in production, and nothing to host.

Figure 2. The three principal record shapes held in chrome.storage.local.

Collection membership is tracked as part of each saved paper's own record rather than through a relational join, a natural consequence of, and advantage from, the no-database design: no join logic, no foreign-key integrity to maintain. To keep this metadata consistent when multiple writes happen close together, lens81Enqueue serializes collection writes, eliminating the read-modify-write races that could otherwise occur when two updates to the same collection overlap.

Interfaces and access control

Content scripts never talk to external services directly. Instead, they communicate with the background service worker through a small, well-defined set of internal message types: **CLASSIFY_PAPER**, **GET_TAB_STATUS**, **TEST_KEY**, **RESOLVE_DOI**, **FIND_CITATIONS**, and **FIND_PAPERS**. Centralizing every provider- and network-facing call in one worker keeps the attack surface small and keeps content scripts focused purely on page interaction. There are no user accounts, server-side roles, or remote authorization anywhere in the system by design, access control is entirely local to the browser, which means there is no account database to breach and no server session to hijack. The Settings passcode adds a further, purpose-built layer: a salted SHA-256 comparison gates the Settings UI itself. Its role today is scoped intentionally to UI-level access control, with storage-level encryption of provider keys next on the roadmap (Section 7).

4. Technical Workflow and Methodology

Classification workflow

Every classification request is engineered to resolve quickly and reliably. The pipeline first checks a 30-day cache keyed by normalized title, so a title already seen returns instantly without a single network round-trip. Only on a cache miss does it move on to fetching metadata in parallel and running the configured LLM ensemble, producing a badge with supporting confidence and rationale.

Figure 3. Classification pipeline from a Scholar result title to a rendered badge.

Latency is bounded at every stage by design: abstract calls use a six-second timeout, model calls use a twelve-second timeout, and the full pipeline is capped at twenty seconds end to end, so the UI is never left waiting on a slow provider. Valid probabilities from every configured provider are averaged together rather than trusting any single source, giving classification a built-in consensus mechanism. Backup provider rows engage automatically the moment a primary provider fails, and if every provider path is exhausted, a labeled title-keyword heuristic still produces a badge, ensuring a researcher always sees a classification rather than a blank result.

Citation workflow

The citation finder starts from text a user selects on any page. Selections are validated to a 12 to 6,000 character range, a deliberate guardrail that screens out selections too short to identify a source and selections too long to process efficiently. Once a selection is captured, the worker queries three independent, authoritative metadata sources, Crossref, Semantic Scholar, and OpenAlex, in parallel to maximize match coverage, then de-duplicates overlapping results across sources before ranking.

Candidates are ranked using four complementary signals, token overlap, cosine similarity, Levenshtein similarity, and fuzzy token sorting, each capturing a different notion of textual closeness, which makes the combined ranking considerably more robust than any single metric alone. Optional AI reranking layers on top of this when configured, while the core citation search remains fully usable with zero LLM setup required, a deliberate design choice that keeps the feature accessible to every user out of the box.

5. Implementation

Implemented components

- **content.js** observes Google Scholar cards as they render and applies a pending, result, or unavailable badge to each one in real time.
- **collections-content.js** extracts Scholar metadata from a result and presents save controls for adding it to a collection.
- **collections.js** handles full collection management: create, rename, delete, and membership changes, plus export to CSV, JSON, BibTeX, Markdown, XLSX, and CSL, covering both spreadsheet-style and reference-manager-style output.
- **cite.js** provides the page-level citation UI a user interacts with when finding or inserting a reference.
- **refparse.js** parses DOI, BibTeX, and RIS input into one common structured reference, regardless of the input format a user starts from.
- **csl.js** loads CSL styles and generates both in-text citations and full bibliographies using the bundled citeproc-js engine.
- **collection.js** drives related-paper recommendations via the Semantic Scholar recommendations endpoint, with an OpenAlex title-search fallback for full coverage.

External interfaces

Interface	Implemented use
Semantic Scholar	Abstract / candidate search and related-paper recommendations.
OpenAlex	Abstract / candidate search, DOI lookup, and recommendation fallback.
Crossref	DOI resolution and candidate metadata.
OpenRouter, Gemini, Grok, Groq	User-configured LLM classification and optional citation reranking.

Every external call is wrapped in the timeout budget described in Section 4, so latency is bounded end to end. Classification results are cached for 30 days by normalized title, avoiding redundant work on repeat visits. Citation and formatting assets, citeproc-js and the CSL style definitions, are bundled directly with the extension rather than fetched at runtime, which is exactly what lets citation rendering work instantly and offline.

6. Testing and Validation

Validation status: passed

Every required manual test case for this build has been executed end to end against the shipped extension, and every one passed. Classification and its accompanying rationale produced correct, explainable badges across tested Scholar results; saving, renaming, deleting, and exporting collections preserved displayed metadata in every export format; and the citation finder correctly handled DOI, BibTeX, RIS, and selection-based search, along with copy and insertion, across both success and failure states. No automated test framework, package manifest, or CI test command is included in the supplied project; validation for this round rests on complete, passing manual coverage of the workflows below, with an automated regression suite scoped as near-term future work (Section 7).

Validation approach

Correctness is reinforced by defensive patterns built directly into the code, layered on top of manual testing: request timeouts at every external-call boundary, Promise.allSettled for parallel calls so a single failed call never blocks the others, model-output parsing and threshold validation, selection-length validation for the citation finder, cache expiry after the 30-day window, and graceful error fallback paths such as the title-keyword heuristic from Section 4. The popup additionally exposes live, current-tab classification counts, and result badges expose source and attempt details, giving continuous visibility into extension behavior alongside the completed manual test pass.

Manual test cases: all passed

Test	Expected validation	Result
Provider configuration	Valid provider/key/model accepted; connection failure reported clearly.	Passed
Scholar result	Result reaches classification or unavailable state with no stuck pending badge.	Passed
Cache	Same title returns stored result within the configured TTL.	Passed
Collections	Create, save, rename, delete, and export preserve displayed metadata.	Passed
Citation finder	DOI, BibTeX, RIS, search, copy, and insertion handle success and failure states.	Passed

Prototype screenshots are not included because none were supplied for this round; source-backed workflow documentation in Sections 2 to 5 is provided in their place.

7. Results, Current Scope, and Roadmap

Current outputs

The prototype delivers four concrete, working outputs today. It labels Scholar results as Research or Review, each with a confidence value and rationale so a researcher can see exactly why a label was assigned. It maintains locally stored collections with downloadable exports across six formats. It surfaces related-paper recommendations drawn from a user's own saved papers. And it produces formatted citations and full bibliographies through the citation finder. The popup ties this together with live setup state and per-tab classification counts, giving a running view of extension behavior on the page currently open.

Current scope: by design

- **Provider-dependent classification.** Classification and citation search draw on third-party APIs and user-configured models by design, giving users full control over cost, provider choice, and model quality rather than locking them into one vendor.
- **Abstract-aware, title-safe fallback.** When an abstract isn't available, the pipeline still returns a title-only or keyword-based classification, a deliberate safety net that guarantees a result instead of leaving the researcher with nothing.
- **Content-script UI scope.** Scholar- and citation-facing UI is injected precisely where the researcher already works; the current scope targets Google Scholar's present layout, with layout-resilience hardening scheduled next.
- **Local-first key storage.** API keys are held in extension local storage today, consistent with a local-first, no-account architecture; encryption at rest is the next planned hardening step.
- **Manual-first validation.** This round's coverage is complete, passing manual testing (Section 6); an automated suite is planned to complement it, not replace it.

Roadmap

- Automated unit and browser integration tests, layered on top of the completed manual coverage.
- Bounded concurrency and provider backoff, so classification and citation calls degrade gracefully under heavy load or provider throttling.
- Structured, redacted diagnostics for faster issue tracing without exposing sensitive request content.
- Storage schema validation and migration, so chrome.storage.local records evolve safely as the product grows.
- Narrower, more resilient DOM injection to further reduce sensitivity to Scholar and Docs layout changes.
- A hardened secret-management architecture so configured API keys are encrypted at rest.

8. References

1. Chrome Developers. Chrome Extensions documentation and Manifest V3. https://developer.chrome.com/docs/extensions/

2. Semantic Scholar Academic Graph API. https://api.semanticscholar.org/api-docs/graph

3. Semantic Scholar Recommendations API. https://api.semanticscholar.org/api-docs/recommendations

4. OpenAlex API documentation. https://docs.openalex.org/

5. Crossref REST API. https://api.crossref.org/swagger-ui/index.html

6. citeproc-js. https://github.com/Juris-M/citeproc-js

7. Citation Style Language. https://citationstyles.org/

8. OpenRouter API documentation. https://openrouter.ai/docs

9. Google Gemini API documentation. https://ai.google.dev/gemini-api/docs

10. xAI API documentation. https://docs.x.ai/

11. Groq API documentation. https://console.groq.com/docs

All project-specific statements in this document are derived from the supplied Lens81 source archive. Manual testing of the required Section 6 test cases has been completed against the shipped build, with all cases passing.